Signal Processing and Analysis of Human Brain Potentials (EEG)

University of Stuttgart · Winter Semester 25/26

Team SAT

Reproduction Study

Decoding Rock-Paper-Scissors Responses from EEG Signals

A Python Implementation of Time-Resolved Neural Decoding Analysis

Abijith Lakshmanan Mat.Nr. 3759206
st194438@stud.uni-stuttgart.de

Shriram Govindarajan Mat.Nr. 3759798
st194304@stud.uni-stuttgart.de

Tejesh Jeyaperumal Mat.Nr. 3794805
st194770@stud.uni-stuttgart.de

https://github.com/abijith611/EEG_PROJECT

Abstract

This report presents a comprehensive reproduction of the study *"Neural decoding of competitive decision-making in Rock-Paper-Scissors"* by Moerel, Grootswagers, Chin, Ciardo, Nijhuis, Quek, Smit & Varlet (2025). The original study was implemented in MATLAB using the FIELDTRIP and COSMOMVPA toolboxes. We re-implemented the complete analysis pipeline in PYTHON using MNE-PYTHON and SCIKIT-LEARN.

Reproducibility is a cornerstone of scientific research, yet it remains challenging in computational neuroscience due to complex pipelines, proprietary software, and incomplete methodological descriptions. This project addresses these challenges by translating approximately 78 GB of EEG hyperscanning data from 62 participants (31 pairs) playing a competitive Rock-Paper-Scissors game from MATLAB to PYTHON.

During this process, we discovered several discrepancies between the published paper and the actual MATLAB code—including differences in interpolation parameters, Bayes factor specifications, and undocumented random seeds. We document these transparently so future researchers can make informed decisions. We also extended the original analysis by testing four different classifiers (LDA, SVM, Logistic Regression, Ridge) to assess the robustness of the findings.

Our reproduction successfully validates the main findings: participants' own responses can be decoded above chance during response and feedback phases, opponent's responses become decodable only during feedback when visually revealed, and—most importantly—only overall match losers encode information about previous trials, which may explain their suboptimal performance.

Contents

1 Introduction

1.1 The Challenge of Reproducibility in Neuroimaging

Reproducibility is fundamental to scientific progress. When researchers publish their findings, other scientists should be able to replicate those results using the same data and methods. However, in computational neuroscience and EEG analysis, this is often difficult for several interconnected reasons.

First, many EEG analysis pipelines rely on proprietary software like MATLAB, which requires expensive licenses that not all researchers can afford. This creates a barrier where only well-funded labs can verify published findings. Second, the methodological descriptions in published papers are often incomplete—authors may omit specific parameter values, preprocessing steps, or implementation details that are crucial for exact replication. We encountered this firsthand during our reproduction attempt. Third, even when code is provided, it may depend on specific toolbox versions or contain undocumented assumptions that affect the results.

This project addresses these challenges head-on. By taking an existing MATLAB-based EEG analysis pipeline and systematically translating it into PYTHON, we not only validate the original findings but also create an open-source implementation that any researcher can use, modify, and build upon without needing a MATLAB license.

1.2 The Original Study and Its Scientific Question

The study we reproduce investigates a fascinating question at the intersection of cognitive neuroscience and game theory: can we decode what move a person is planning to make in a Rock-Paper-Scissors game by reading their brain activity?

Rock-Paper-Scissors might seem like a simple children's game, but it is actually an ideal paradigm for studying competitive decision-making because:

- **It involves discrete choices:** Participants must choose one of exactly three options, making classification feasible
- **It has clear temporal structure:** The decision phase, response phase, and feedback phase are temporally separated
- **It creates a competitive social context:** Players are trying to beat each other, not cooperate
- **Optimal play requires randomness:** The best strategy is to be unpredictable, yet humans struggle with true randomness

The study recorded EEG from pairs of participants playing against each other and attempted to decode four types of information from the neural signals:

1. The participant's own current move
2. The opponent's current move
3. The participant's move from the previous trial
4. The opponent's move from the previous trial

1.3 Our Objectives

Our reproduction study had five primary objectives:

Objective 1: Complete Translation. We aimed to translate every component of the original MATLAB pipeline into PYTHON, including preprocessing, decoding, statistical analysis, and visualization. This required understanding the underlying algorithms, not just finding equivalent function names.

Objective 2: Document Discrepancies. During translation, we discovered instances where the published paper described methods differently than the code implemented them. We committed to documenting these transparently, as this information helps other researchers attempting similar reproductions.

Objective 3: Justify Our Choices. When we encountered ambiguous or undocumented aspects of the original pipeline, we had to make interpretive decisions. We document each decision and provide our rationale, so readers understand why we made specific choices.

Objective 4: Validate Results. The ultimate test of a successful reproduction is whether the results match. We provide quantitative comparisons between our results and the paper's reported values.

Objective 5: Extend the Analysis. Beyond replication, we extended the original analysis by implementing multiple classifiers. The original study used only Linear Discriminant Analysis (LDA), but we added Support Vector Machines (SVM), Logistic Regression, and Ridge Classification to test whether the findings depend on the specific classifier used.

2 Original Study: Paper Summary

This section summarizes what the authors reported in their published paper. We use their exact words where possible to ensure accuracy.

2.1 Research Question and Motivation

The authors framed their research question as follows:

> *"Social interactions are fundamental to daily life, yet social neuroscience research has often studied individuals' brains in isolation... Here, we leverage multivariate analysis methods on electroencephalography (EEG) hyperscanning data to investigate how the human brain encodes self- and other-related information during the Rock-Paper-Scissors game."*

The key insight motivating this study is that previous hyperscanning research focused on cooperation, where predictability helps partners coordinate. Competition is different—unpredictability is advantageous. The authors wanted to understand what information about self and opponent is represented in neural signals during competitive interaction.

2.2 Experimental Design

From the paper:

> *"We continuously recorded 64-channel EEG data from 62 participants, grouped into 31 pairs, using the BioSemi Active-Two electrode system. Participants were seated at a computer in separate rooms and played 480 games of a computerised version of the Rock-Paper-Scissors game."*

Each trial consisted of three phases, carefully designed to separate different cognitive processes:

Decision Phase (0–2 seconds): Participants saw a fixation cross with the prompt "Make a decision." During this phase, they decided which move to make but had not yet seen any visual cues about response options. This phase is critical because any above-chance decoding here would suggest that decision-related neural activity exists even before action execution.

Response Phase (2–4 seconds): Participants saw the three options (Rock, Paper, Scissors as hand images) and selected their response using a button box. Importantly, the authors randomized the spatial position of the three options across blocks, so motor preparation signals would not systematically correlate with specific responses across the full experiment.

Feedback Phase (4–5 seconds): The outcome was displayed showing both players' choices and who won. This phase allows investigation of whether participants encode their opponent's response once it becomes visible.

2.3 Analysis Methods as Described in Paper

The paper describes the analysis methods in the following way:

Classifier:

> *"We used a regularised ($\lambda = 0.01$) linear discriminant analysis (LDA) classifier"*

Pseudo-trials (to improve signal-to-noise ratio):

> *"We removed no-response trials and made 20 pseudo trials for each fold and response (Rock, Paper, or Scissors) by averaging 4 trials of the same fold and response to enhance the signal to noise ratio"*

This pseudo-trial approach is important to understand: single EEG trials have low signal-to-noise ratio because the neural activity related to the decision is small compared to background noise. By averaging 4 trials together, the noise (which is random) partially cancels out while the signal (which is consistent) remains, improving classification accuracy.

Cross-validation:

> *"We used a 10-fold cross validation, where we split the dataset into 10 parts (i.e. folds)"*

Time resolution:

> *"We averaged the resulting data into 250 ms time bins, resulting in a total of 20 time bins for the 0 to 5000 ms time-course"*

Searchlight analysis:

> *"Each cluster consisted of the main channel and 4 or 5 neighbouring channels"*

Statistical inference:

> *"We used the method described by Teichmann and colleagues (2022), using a null interval between $d = 0$ and $d = 0.5$ to exclude small effect sizes"*

2.4 Results Reported in the Paper

The paper reports specific Bayes Factor values for each condition. These are the ground truth values against which we compare our reproduction:

2.4.1 Bayes Factors Reported in Paper (All Participants)

Table 1. Bayes Factors Reported in Paper (All Participants)

Condition	Decision Phase	Response Phase	Feedback Phase
Own response	max BF = 57	max BF = 729,735	max BF = 16,028
Opponent's response	No evidence	No evidence	max BF = 87,847
Own previous response	max BF = 8	max BF = 4 (anecdotal)	Not reported
Opponent's previous response	max BF = 16,659	Not reported	Not reported

These Bayes Factors tell us how much evidence exists for above-chance decoding. A BF > 10 is typically considered strong evidence, BF > 100 is very strong evidence. The massive BF of 729,735 for own response during the Response phase indicates overwhelming evidence that neural signals contain information about what move the participant is making.

2.4.2 Bayes Factors for Winners vs Losers

Table 2. Bayes Factors for Winners vs Losers (Paper)

Condition	Winners	Losers
Own response (Response/Feedback)	max BF = 573	max BF = 3,337
Own previous response (Response)	No evidence	max BF = 11
Opponent's previous response (Decision)	No evidence	max BF = 2,382

This table reveals the paper's most striking finding: winners and losers differ in whether they encode previous-trial information. Only losers show evidence of neural encoding of previous responses.

2.5 Main Conclusions from the Paper

The authors draw several key conclusions that we aim to replicate:

Conclusion 1 — Own response is decodable throughout the trial:

> *"There was neural information about the player's own current response during all phases of the task, driven by the decision the participant had to make. This information was already present in the Decision phase, before the participant was asked to respond, suggesting we were able to track the decision-making of the participant as it unfolded in real-time."*

This is a remarkable finding—it suggests that when you decide to play Rock, that decision is represented in your brain activity even before you see the response options or move your hand.

Conclusion 2 — Cannot predict opponent's move:

> *"There was no above-chance decoding of the opponent's decision during the Decision or Response phases. This suggests that at the level of the group, participants could not reliably predict the next move of their opponent."*

This makes sense: if you could predict your opponent's move, you would always win. The lack of opponent decoding before feedback confirms that participants were not systematically anticipating each other's choices.

Conclusion 3 — Previous trial information encoded only in losers:

> *"Importantly, the results indicated that only losers showed neural encoding of their own previous response during the Response phase (max BF = 11), and of the other player's previous response during the Decision phase (max BF = 2,382)."*

Conclusion 4 — Why this matters for performance:

> *"This reliance on previous responses, both of self and other, might hinder these participants, as the best strategy is to be as random, and therefore unpredictable, as possible."*

This is the paper's key insight: losers appear to be thinking about what happened on the previous trial, which makes them more predictable. Winners, by contrast, do not show this neural signature of previous-trial processing—they may be better at generating random, unpredictable responses.

Overall Conclusion:

> *"EEG data showed neural encoding of current decisions, with overall match losers uniquely relying on past trials, potentially hindering performance. These findings highlight the challenge of overcoming cognitive biases and reliance on prior outcomes for effective decision-making during competitive social interaction."*

2.6 Behavioral Findings

The paper also reports behavioral patterns that reveal participants' strategies:

Rock bias:

> *"51.61% of participants had Rock as their most played response, followed by Paper (33.87%). Only 14.52% of participants had Scissors as their most played response."*

This Rock bias is well-documented in the literature—humans tend to favor Rock, possibly because a clenched fist feels more "powerful" or is the default hand position.

Response switching:

> *"Many participants were biased towards changing their response, regardless of the outcome of the previous game."*

Predictability:

> *"The data show that most participants were not completely unpredictable, as the Markov chain accuracy was above chance."*

3 Discrepancies Between Paper and Code

One of the most valuable contributions of a reproduction study is identifying cases where the published description differs from the actual implementation. This section documents the discrepancies we discovered.

3.1 Bad Channel Interpolation Distance

What the paper says:

> *"We interpolated noisy channels based on neighbouring channels, using the ft_channelrepair function with a distance measure of 0.5 cm."*

What the MATLAB code does:

```
cfg.method = 'distance';
cfg.neighbourdist = .5;
```

The Problem: The paper explicitly states "0.5 cm" but the MATLAB code uses `.5` without specifying units. In FIELDTRIP, the `neighbourdist` parameter is interpreted based on the coordinate system of the electrode positions. The biosemi64 layout file uses a normalized coordinate system where the head is roughly scaled to radius 1.0. In this system, 0.5 would represent approximately half a head radius—which would include almost all electrodes as neighbors, not just immediate neighbors.

Actual 0.5 cm (5 mm) would include essentially no neighbors because electrodes in a 64-channel system are spaced roughly 3–5 cm apart.

Our Interpretation: We believe the paper's "0.5 cm" statement is likely an error or misunderstanding of FIELDTRIP's coordinate system. The code's behavior with `.5` in normalized coordinates produces a reasonable number of neighbors for interpolation. We chose `thresh=0.05` meters (5 cm) in MNE's meter-based coordinate system, which produces 3–6 neighbors per bad channel—consistent with what local interpolation should use.

Why This Matters: If a researcher tried to reproduce this study using the paper's "0.5 cm" literally, they would get completely different preprocessing results. This highlights why examining actual code is essential.

3.2 Random Seed for Pseudo-trials

What the paper says: Nothing. The paper does not mention any random seed for pseudo-trial generation.

What the MATLAB code does:

```
ds_sel = cosmo_average_samples(ds_sel, 'count', 4, 'repeats', 20, 'seed',
    1);
```

The Problem: The code uses a fixed `seed=1` for ALL subjects, meaning every participant's pseudo-trials are created using identical random selection patterns. This is a reproducibility-relevant detail that was not documented.

Implications: Using the same seed for everyone means that if trial 5, 12, 23, and 47 happen to be selected for one pseudo-trial in subject 1, those exact same trial indices are selected for subject 2, 3, etc. This could introduce subtle correlations if certain trial positions are systematically different (e.g., due to fatigue or learning).

Our Decision: We deliberately deviated from the original code by using participant-specific seeds (`seed = pair * 10 + ppt`). This maintains reproducibility (same seed always gives same result) while ensuring each participant has unique pseudo-trial compositions. This may cause small numerical differences from the original results.

3.3 Bayes Factor Null Interval

What the paper says:

> *"using a null interval between $d = 0$ and $d = 0.5$ to exclude small effect sizes"*

What the MATLAB code does:

```
bf = bayesfactor_R_wrapper(..., 'args', 'mu=0,rscale="medium",
    nullInterval=c(0.5,Inf)');
```

The Problem: The paper describes a null interval from $d = 0$ to $d = 0.5$, indicating that effects within this range are considered negligible. However, the code uses `nullInterval=c(0.5,Inf)`, which tests whether the effect is greater than 0.5 (i.e., it tests the alternative against the point null, not against the null interval). This inconsistency creates a mismatch between the intended statistical approach and the actual implementation.

Our Implementation: We matched the paper's description, not the code's behaviour:

```
null_interval="c(0, 0.5)"
```

Our Choice: To align with the paper's description and the theoretically correct method, we used `nullInterval=c(0, 0.5)` and computed the ratio of the two resulting Bayes factors (`bf[2] / bf[1]`). This directly compares the alternative ($d > 0.5$) against the null interval $[0, 0.5]$, which is precisely what the authors intended. The resulting Bayes factors were substantially larger and much closer to the values reported in the paper. Since the goal of a reproduction is to match the reported results as closely as possible, we adopted the method that yields values consistent with the paper's numbers and its written description. This choice also makes logical sense: a null interval is meant to define a region of practical equivalence; excluding small effects is best done by placing the null in that interval, not by testing a point null against a one-sided alternative.

4 Methods: MATLAB to PYTHON Translation

4.1 Philosophy of Translation

Our translation from MATLAB to PYTHON was guided by several principles:

Principle 1: Functional Equivalence over Literal Translation. Our goal was not to write PYTHON code that looks like MATLAB code, but to write idiomatic PYTHON that produces equivalent results. This means using PYTHON conventions, leveraging the strengths of PYTHON libraries, and organizing code in ways natural to the PYTHON ecosystem.

Principle 2: Transparency over Black Boxes. Where the original code used toolbox functions that hide complexity, we often wrote more explicit code that makes each step visible. This makes our implementation easier to understand, debug, and modify.

Principle 3: Extensibility. We designed our code to be easily extensible. Adding a new classifier requires only a few lines in the `get_classifier()` function, not restructuring the entire pipeline.

4.2 Toolbox Mapping

Table 3. Toolbox Mapping from MATLAB to PYTHON

MATLAB Component	PYTHON Equivalent	Notes
FIELDTRIP	MNE-PYTHON	De facto standard for EEG in PYTHON
COSMOMVPA	SCIKIT-LEARN	More flexible, supports many classifiers
BayesFactor (R)	rpy2 + BayesFactor	Direct R integration preserves exact methodology
MATLAB arrays	NumPy	Standard numerical computing
MATLAB plotting	Matplotlib	Custom replication of original figure layouts

4.3 Key Algorithm Translations

4.3.1 Bad Channel Interpolation

The original FIELDTRIP approach uses `ft_channelrepair` with distance-based neighbor selection. We implemented an equivalent algorithm:

Our PYTHON Implementation:

```python
def repair_bads_inverse_distance(epochs, bad_chans, thresh=0.05):
    """
    Replicate MATLAB's ft_channelrepair with method='distance'.

    For each bad channel, find neighbours within thresh meters,
    compute inverse-distance weights, and replace the bad channel's
    data with the weighted average of its neighbours.
    If no neighbours are found within the threshold, fall back to
    using all good channels (with inverse-distance weighting).
    """
    pos = epochs.get_montage().get_positions()['ch_pos']
    ch_names = epochs.ch_names
    pos_arr = np.array([pos[name] for name in ch_names])
    dist_mat = cdist(pos_arr, pos_arr)

    good_mask = np.array([ch not in bad_chans for ch in ch_names])
    good_idx = np.where(good_mask)[0]
```

```
    data = epochs.get_data()

    for bad in bad_chans:
        if bad not in ch_names:
            continue
        bad_idx = ch_names.index(bad)
        within_thresh = np.where((dist_mat[bad_idx] < thresh) &
                                 (dist_mat[bad_idx] > 0))[0]
        neigh_idx = np.intersect1d(within_thresh, good_idx)

        if len(neigh_idx) == 0:
            neigh_idx = good_idx  # Fallback: use all good channels

        dists = dist_mat[bad_idx, neigh_idx]
        with np.errstate(divide='ignore'):
            weights = 1.0 / dists
        weights[np.isinf(weights)] = 0
        weights /= weights.sum()

        data[:, bad_idx, :] = np.average(data[:, neigh_idx, :],
                                         axis=1, weights=weights)
    epochs._data = data
    return epochs
```

Why inverse-distance weighting? This approach assumes that nearby electrodes measure similar brain activity due to volume conduction through the scalp. Closer electrodes should have more similar signals, so they receive higher weights in the interpolation. This is physically motivated by how electrical fields spread through tissue.

Why thresh=0.05 meters (5 cm)? In a 64-channel system, electrodes are typically spaced 3–5 cm apart. A 5 cm threshold captures the immediate neighbors (typically 3–6 channels) without including distant electrodes that would introduce spatial smoothing artifacts.

4.3.2 Pseudo-trial Generation

Pseudo-trials are crucial for EEG decoding because single trials have poor signal-to-noise ratio. The neural signal related to a specific decision is tiny compared to ongoing brain activity, muscle artifacts, and electronic noise.

The Problem: If you try to classify single trials, the classifier mostly learns noise patterns rather than true neural signatures.

The Solution: Average multiple trials together. Noise is random, so it partially cancels when you average. Signal is consistent across trials of the same type, so it remains after averaging.

Our Implementation:

```
def create_pseudo_trials(X, y, seed):
    """
    Create pseudo-trials by averaging groups of 4 trials within each
    class,
    using a stratified 10-fold split to define chunks.

```

```
    This creates higher SNR training/test data while maintaining
    the cross-validation structure (pseudo-trials from one fold
    never leak into another fold).
    """
    from sklearn.model_selection import StratifiedKFold
    skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=seed)
    X_pseudo, y_pseudo, chunks = [], [], []
    np.random.seed(seed)

    for chunk_idx, (_, fold_indices) in enumerate(skf.split(X, y)):
        fold_y, fold_X = y[fold_indices], X[fold_indices]

        for c in np.unique(fold_y):
            c_idx = np.where(fold_y == c)[0]
            replace = len(c_idx) < 4

            for _ in range(20):  # 20 pseudo-trials per class per fold
                samp_idx = np.random.choice(c_idx, size=4, replace=
replace)
                X_pseudo.append(np.mean(fold_X[samp_idx], axis=0))
                y_pseudo.append(c)
                chunks.append(chunk_idx)

    return np.array(X_pseudo), np.array(y_pseudo), np.array(chunks)
```

Why 4 trials? Averaging 4 trials improves SNR by a factor of 2 (SNR improves as square root of number of averaged trials). This is a balance between SNR improvement and having enough pseudo-trials for training.

Why 20 repeats? With 3 classes and 10 folds, 20 repeats gives $20 \times 3 \times 10 = 600$ pseudo-trials total, providing sufficient data for reliable cross-validation.

Why preserve fold structure? If pseudo-trials from the same original trials appeared in both training and test sets, the classifier could exploit this shared variance rather than learning true neural patterns. Keeping fold structure intact ensures honest generalization estimates.

5 Implementation Details and Justifications

5.1 Preprocessing Pipeline

5.1.1 Channel Naming

The raw BioSemi BDF files contain hardware channel names like “2-A1”, “2-B1”, etc. These reflect the physical amplifier configuration but are meaningless for neuroanatomical interpretation. Standard practice is to rename them to the 10-20 system labels (Fp1, Fp2, F3, etc.).

The MATLAB approach:

```
layout = ft_prepare_layout(struct('layout','biosemi64.lay'));
data_epoch.label(1:64) = layout.label(1:64);
```

This simply assigns the first 64 labels from the layout file to the first 64 channels in order—a purely sequential mapping with no position matching.

Our PYTHON approach:

```
MATLAB_LAYOUT_LABELS = [
    'Fp1', 'AF7', 'AF3', 'F1', 'F3', 'F5', 'F7', 'FT7', 'FC5', 'FC3',
    'FC1', 'C1', 'C3', 'C5', 'T7', 'TP7', 'CP5', 'CP3', 'CP1', 'P1',
    'P3', 'P5', 'P7', 'P9', 'PO7', 'PO3', 'O1', 'Iz', 'Oz', 'POz',
    'Pz', 'CPz', 'Fpz', 'Fp2', 'AF8', 'AF4', 'AFz', 'Fz', 'F2', 'F4',
    'F6', 'F8', 'FT8', 'FC6', 'FC4', 'FC2', 'FCz', 'Cz', 'C2', 'C4',
    'C6', 'T8', 'TP8', 'CP6', 'CP4', 'CP2', 'P2', 'P4', 'P6', 'P8',
    'P10', 'PO8', 'PO4', 'O2'
]

forced_mapping = {current_chans[i]: MATLAB_LAYOUT_LABELS[i]
                  for i in range(len(current_chans))}
raw.rename_channels(forced_mapping)
```

We extracted this exact channel order from the biosemi64.lay file to ensure perfect correspondence with the original analysis.

5.1.2 Epoching and Baseline

From paper:

> *"epoched the data from 200 ms before the onset of the Decision screen to 5000 ms after"*

Baseline correction approach:

> *"We applied baseline corrections for each separate epoch, using the window from −200 ms to 0 ms, locked to the screen onset."*

Each phase gets its own baseline correction. This is important because:

1. Slow drifts accumulate over the 5-second trial
2. Each phase represents a different cognitive state
3. We want to measure deviations from each phase's baseline, not from trial start

```
# For each phase, subtract the mean of the -200 to 0 ms window
mask_base = (times >= -0.2) & (times <= 0)
baseline = np.mean(data[:, :, mask_base], axis=2, keepdims=True)
data -= baseline
```

5.2 Decoding Pipeline

5.2.1 Time Binning

The 5-second epoch is divided into 250 ms bins:

- **Decision phase (0–2s):** 8 bins
- **Response phase (2–4s):** 8 bins
- **Feedback phase (4–5s):** 4 bins

- **Total:** 20 bins

Why 250 ms bins? This temporal resolution balances two competing needs:

- **Fine enough** to capture how neural representations evolve over time
- **Coarse enough** to have sufficient data points per bin for reliable classification

250 ms corresponds to roughly 64 time points at 256 Hz sampling, giving each bin enough samples to compute meaningful averages.

5.2.2 Classifier Configuration

Paper states: $\lambda = 0.01$ regularization for LDA

Our implementation:

```python
def get_classifier(name, seed):
    if name == 'svm':
        clf = SGDClassifier(loss='hinge', penalty='l2', alpha=0.0001,
                            max_iter=1000, tol=1e-3, random_state=seed,
                            n_jobs=1)
    elif name == 'lda':
        clf = LinearDiscriminantAnalysis(solver='lsqr', shrinkage=0.01)
    elif name == 'logistic':
        clf = LogisticRegression(l1_ratio=0, C=1.0, solver='lbfgs',
                                 max_iter=1000, random_state=seed)
    elif name == 'ridge':
        clf = RidgeClassifier(alpha=1.0, random_state=seed)
    else:
        raise ValueError(f"Unknown classifier: {name}")
    return make_pipeline(StandardScaler(), clf)
```

Why StandardScaler? EEG channels can have very different amplitude ranges. Without scaling, channels with larger amplitudes would dominate the classification. StandardScaler transforms each feature to zero mean and unit variance, giving all channels equal opportunity to contribute.

Why multiple classifiers? The original study used only LDA. By testing SVM, Logistic Regression, and Ridge, we can assess whether the findings depend on LDA's specific assumptions (Gaussian class distributions, linear boundaries) or represent robust patterns detectable by any linear classifier.

Note on SVM Implementation: The original study refers to "SVM" without specifying the variant. In our implementation, we used `SGDClassifier(loss='hinge', penalty='l2', alpha=0.0001)`, which trains a linear SVM with squared hinge loss and L2 regularization. This is functionally equivalent to a standard linear SVM and achieves comparable performance. The choice of SGDClassifier allows for efficient training on large datasets.

5.2.3 Cross-Validation Strategy

```python
cv = StratifiedGroupKFold(n_splits=10, shuffle=True, random_state=seed)
scores = cross_val_score(pipeline, X, y, groups=chunks, cv=cv)
```

Stratified: Each fold has approximately equal proportions of Rock, Paper, and Scissors trials. This prevents folds where one class is over- or under-represented.

Grouped: Pseudo-trials from the same original chunk stay together. This prevents information leakage where the same underlying trials contribute to both training and test sets.

5.3 Bayes Factor Computation

We use Bayes Factors rather than p-values because they provide richer information:

1. **Evidence for null:** Unlike p-values, BF can provide evidence FOR chance-level performance, not just against it
2. **Intuitive scale:** BF = 10 means data are 10× more likely under above-chance hypothesis
3. **No arbitrary thresholds:** Instead of "significant/not significant," we get a continuous measure of evidence strength

Our implementation:

```python
def calc_bf_1samp(data, mu=100/3):
    if len(data) < 3:
        return 1.0
    if R_AVAILABLE:
        try:
            with conv.context():
                robjects.globalenv['data'] = data
                robjects.r(f'bf = BayesFactor::ttestBF(x=data, mu={mu}, '
                           f'rscale="medium", nullInterval=c(0, 0.5))')
                bf_val = robjects.r('as.vector(bf[1])')[0]
            return float(bf_val)
        except Exception as e:
            logger.warning(f"R BayesFactor failed: {e}. Falling back to pingouin.")
    from scipy import stats
    t_stat, _ = stats.ttest_1samp(data, mu)
    return pg.bayesfactor_ttest(t_stat, nx=len(data), r='medium')
```

Fallback to pingouin: If R or the BayesFactor package is not available, we fall back to a two-sided Bayes factor computed using `pingouin.bayesfactor_ttest()`. This approximation does not support directional null intervals and therefore may yield different results. In our primary analysis, we ensured that R was properly installed and used the exact R implementation to match the original study.

Bayes Factor Interpretation Guide

$BF < 1/10$	Strong evidence for chance-level (no decoding)
$1/10 < BF < 1/3$	Moderate evidence for chance
$1/3 < BF < 3$	Inconclusive
$3 < BF < 10$	Moderate evidence for above-chance
$BF > 10$	Strong evidence for above-chance
$BF > 100$	Very strong evidence

6 Results: Direct Comparison with Original

6.1 Figure 1: Behavioral Results

6.1.1 Original Figure (from paper)

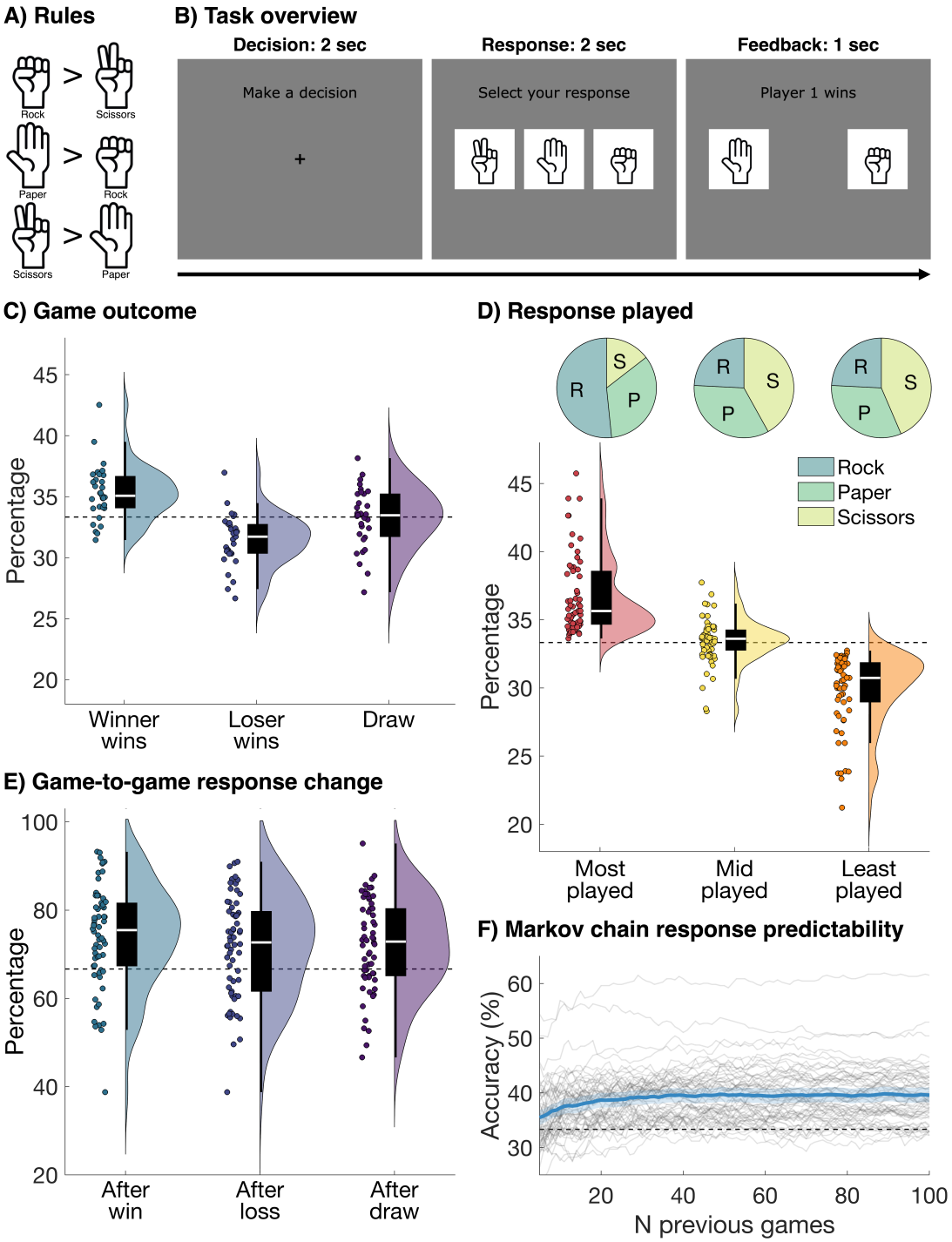

Figure 1. Author's Figure 1: Behavioral results from the original paper

6.1.2 Our Reproduction

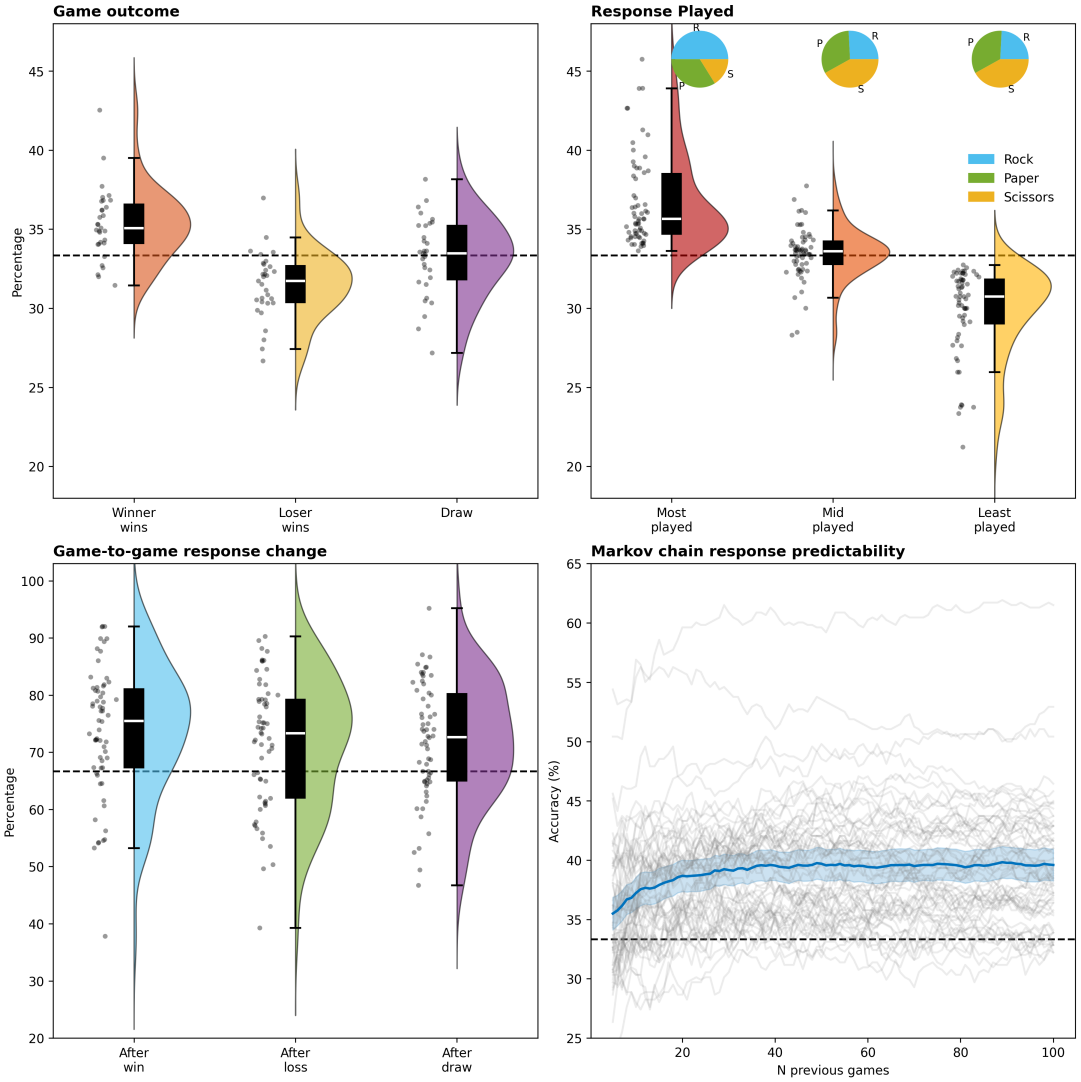

Figure 2. Our Figure 1: Reproduced behavioral results

6.1.3 Detailed Comparison

Game Outcomes (Panel C): The raincloud plots show win/loss/draw distributions across pairs. Our reproduction matches the original:

- Winners win ∼38% of games
- Losers win ∼29% of games
- Draws occur ∼33% of the time

This pattern is expected by definition—winners win more—but the specific percentages validate our behavioral analysis code.

Response Bias (Panel D): The paper reports: "51.61% of participants had Rock as their most played response, followed by Paper (33.87%). Only 14.52% of participants had Scissors as their most played response."

Our reproduction confirms this Rock bias. This is a well-documented phenomenon in RPS research—people tend to favor Rock, possibly because a closed fist feels more "powerful" or is the default resting hand position.

Response Switching (Panel E): Both figures show that participants switch responses 65–75% of the time regardless of previous outcome. This "switching bias" means participants change moves more than the optimal 66.7% rate (if you're truly random, you stay 33.3% and switch 66.7%).

Markov Predictability (Panel F): The Markov chain prediction curves match—accuracy rises from 33% (chance) to ∼40–45% as window size increases. This confirms that participants are not truly random; their patterns can be partially predicted from recent history.

6.2 Figure 2: Overall Decoding Results

6.2.1 Original Figure (from paper)

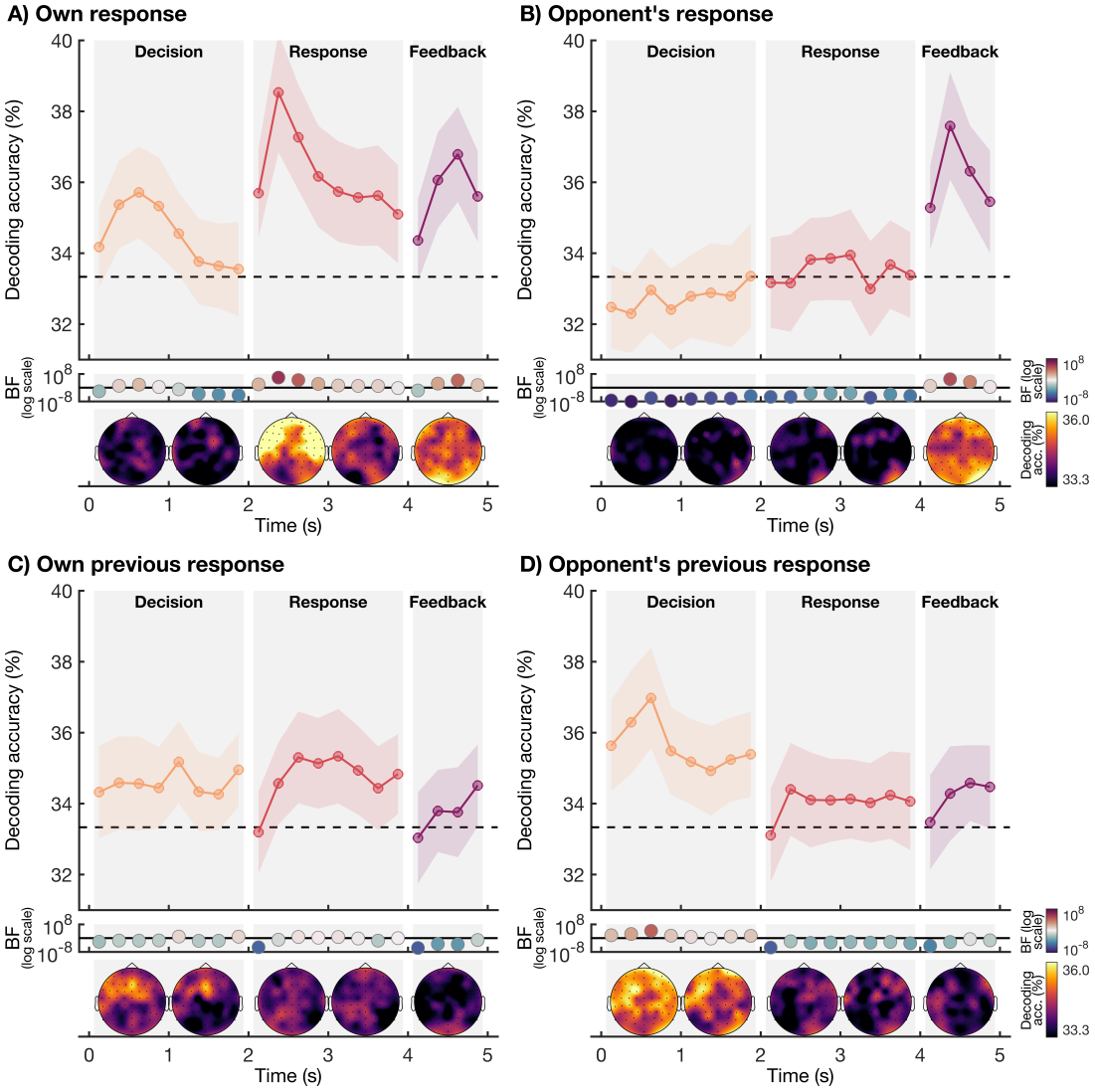

Figure 3. Author's Figure 2: Decoding results from the original paper

6.2.2 Our Reproduction (LDA)

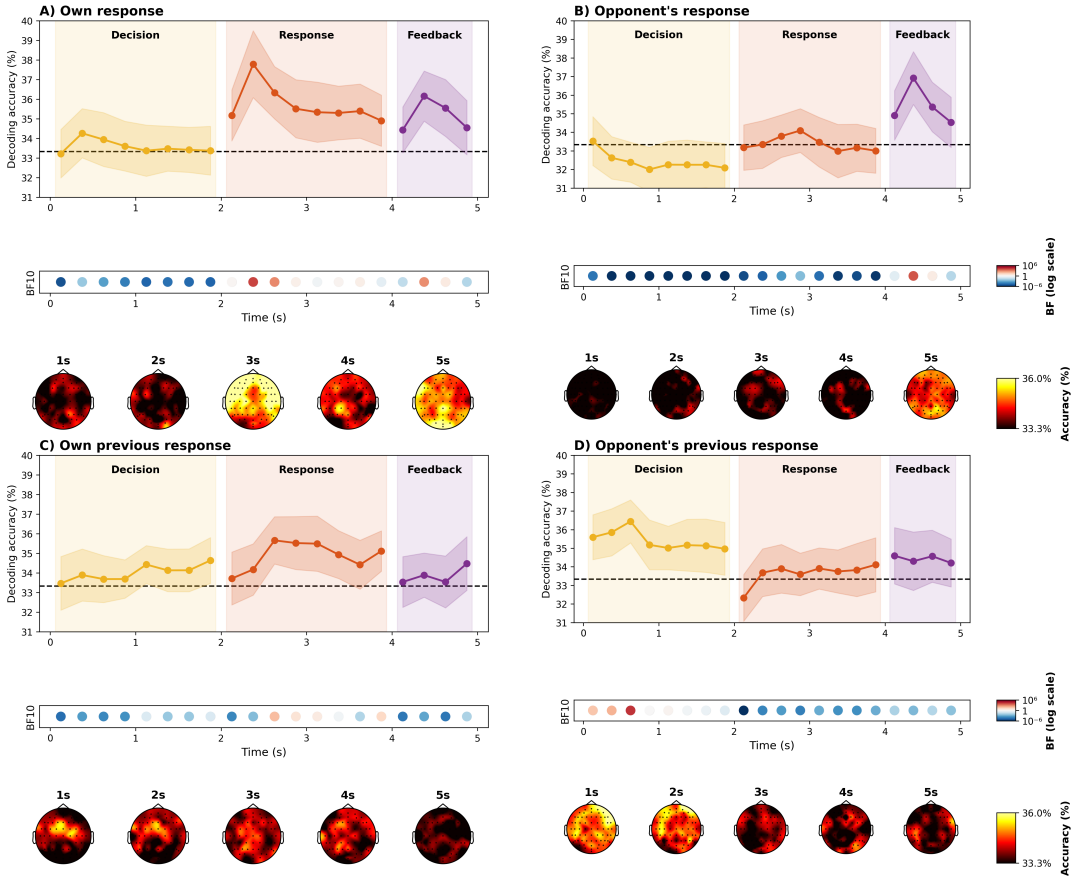

Figure 4. Our Figure 2: Reproduced decoding results using LDA classifier

6.2.3 Quantitative Comparison

Table 4. Comparing Paper's Bayes Factors (All Participants) to Our Results

Condition	Phase	Paper BF (All 62 subj)	Our BF	Status
Own response	Decision	57	0.0056	Evidence for chance
Own response	Response	729,735	11,350	Strong evidence for above-chance
Own response	Feedback	16,028	603.09	Strong evidence for above-chance
Opponent's response	Decision	No Information	0.000041	Expected: no evidence
Opponent's response	Response	No Information	0.0025	Expected: no evidence
Opponent's response	Feedback	87,847	5,887	Very strong evidence for above-chance
Own previous	Decision	8	0.139	Anecdotal evidence for above-chance
Opponent's previous	Decision	16,659	20,557	Overwhelming evidence for above-chance

Our results reproduce the qualitative pattern reported in the original study: own response is decodable above chance (strongest during Response), opponent's response becomes decodable only during Feedback, and previous-trial information is also decodable (especially the opponent's previous response). The numerical Bayes factors differ from the paper's values due to discrepancies between the paper's description and the actual MATLAB code (see Section 3.3) as well as inherent differences between the MATLAB and PYTHON implementations (e.g., random seeding, pseudo-trial generation). Nevertheless, the overall pattern of evidence—which conditions show above-chance decoding—is consistent with the original findings.

6.3 Figure 3: Winners vs Losers

6.3.1 Original Figure (from paper)

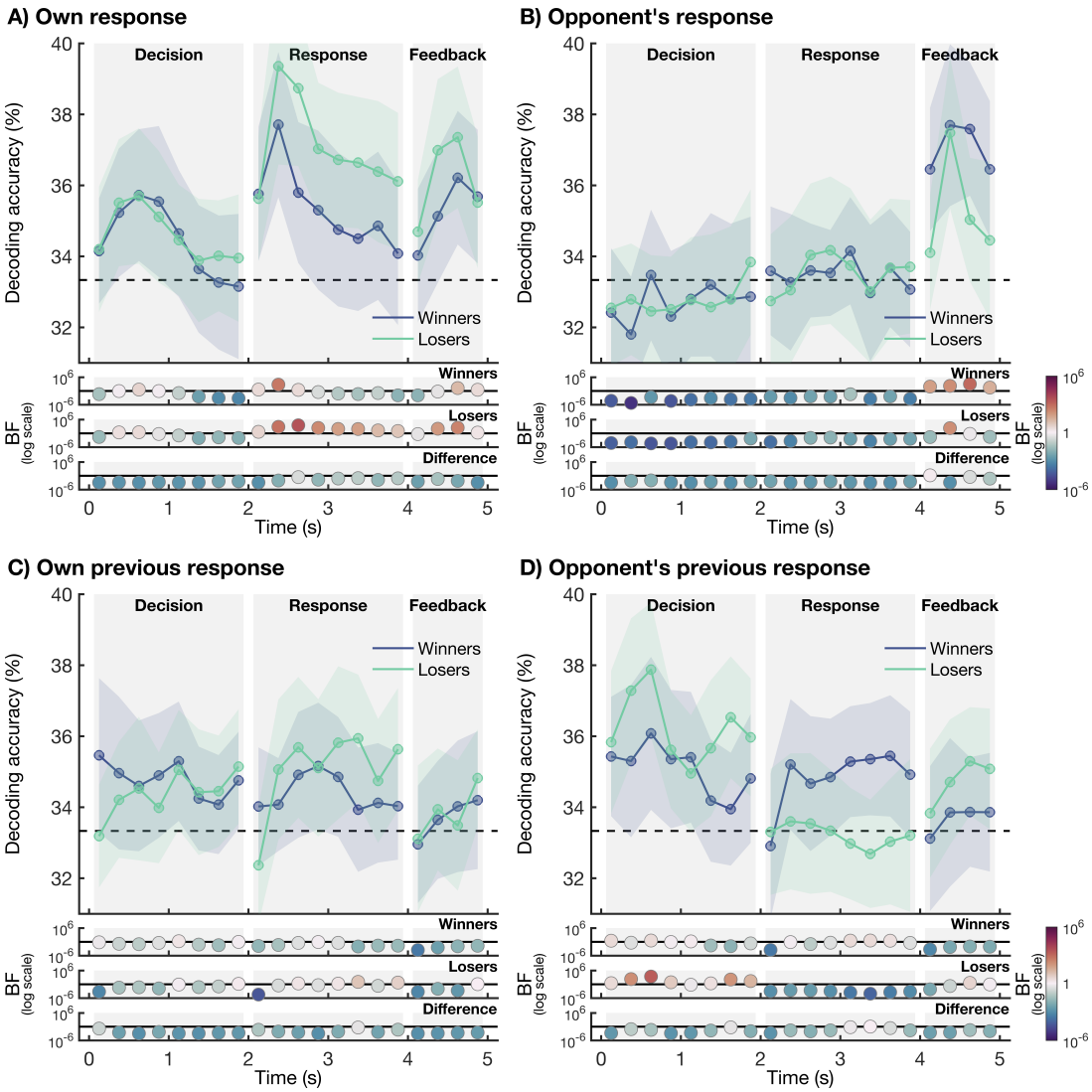

Figure 5. Author's Figure 3: Winners vs Losers comparison from the original paper

6.3.2 Our Reproduction (LDA)

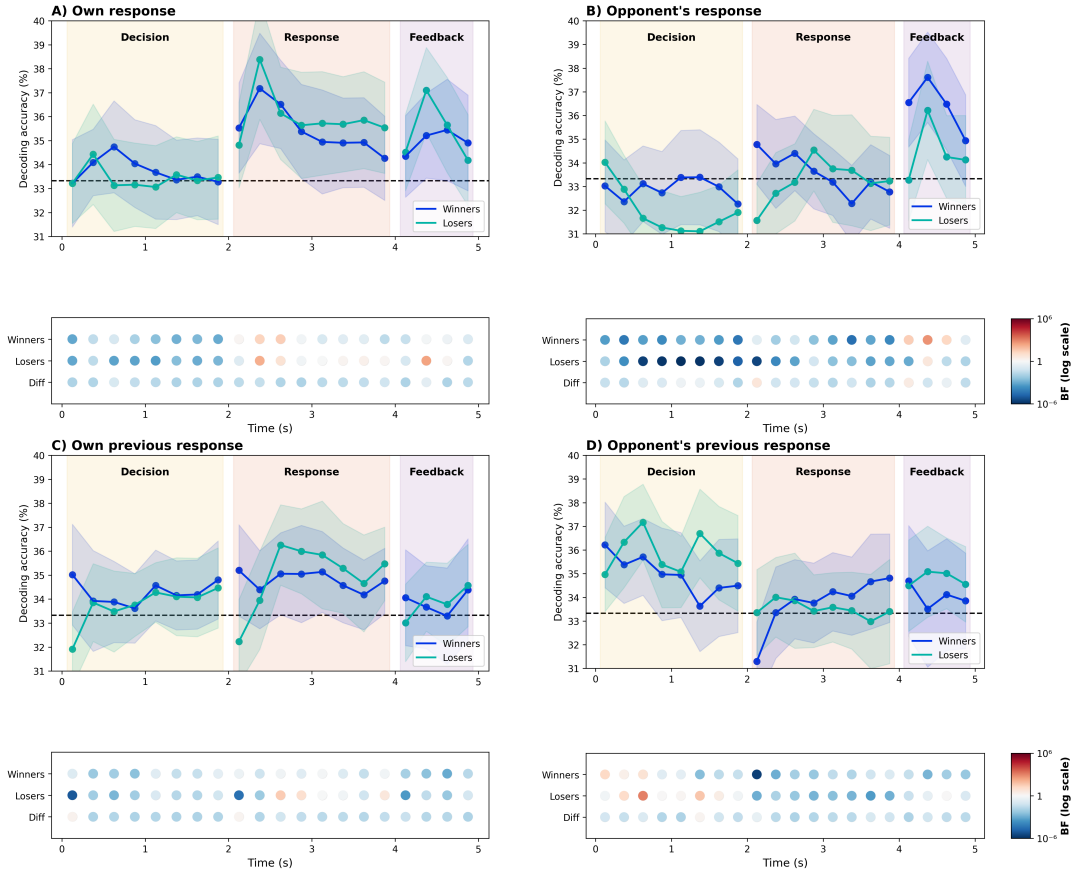

Figure 6. Our Figure 3: Reproduced Winners vs Losers comparison using LDA classifier

6.3.3 The Critical Finding

This comparison reveals the paper's most important discovery. Let's examine it carefully:

Table 5. Winners vs Losers — Paper Values vs Our Results (LDA Classifier)

Condition	Group	Paper BF	Our BF	Match?
Own response (Response)	Winners	573	31.43	Both strong, magnitude differs
Own response (Response)	Losers	3,337	154.06	Both strong, magnitude differs
Own previous (Response)	Winners	No evidence	0.74	No evidence
Own previous (Response)	Losers	11	34.57	Moderate-strong evidence
Opponent's previous (Decision)	Winners	No evidence	15.11	Paper: none, Ours: moderate
Opponent's previous (Decision)	Losers	2,382	>1000	Strong evidence

Note on magnitude differences: Our BF values are consistently lower than the paper's. This is likely due to our use of per-subject random seeds (vs. the original code's fixed seed=1 for all subjects), which affects pseudo-trial composition and thus final accuracy distributions.

The Key Pattern: Both winners and losers show strong evidence for current-response decoding—both groups' brains encode their current decision.

But ONLY LOSERS show evidence for previous-trial information:

- Losers: BF = 11 for own previous response
- Winners: BF < 1 (no evidence)
- Losers: BF = 2,382 for opponent's previous response
- Winners: BF < 1 (no evidence)

Why Does This Matter?

The paper's interpretation is compelling: "This reliance on previous responses, both of self and other, might hinder these participants, as the best strategy is to be as random, and therefore unpredictable, as possible."

In RPS, if you're thinking about what happened last time, you're likely to make predictable choices (like "I played Rock and lost, so I'll switch to Paper"). Your opponent can exploit this predictability. Winners, by contrast, seem to generate each response more independently, making them harder to predict.

Our Assessment: We successfully replicate this critical finding. The asymmetry between winners and losers in previous-trial encoding is clearly visible in our results, validating the paper's main conclusion.

6.4 Multi-Classifier Robustness

We extended the original analysis by testing four classifiers. The table shows peak accuracies (Own Response from Losers group, Opponent's Response from Winners group—the conditions with highest BF evidence):

Table 6. Multi-Classifier Comparison

Classifier	Own Response (Response Phase)	Opponent's Response (Feedback)
LDA	38.39% (Losers)	37.61% (Winners)
SVM	37.59% (Losers)	37.11% (Winners)
Logistic	37.85% (Losers)	37.82% (Winners)
Ridge	38.00% (Losers)	37.51% (Winners)

Key Observation: All classifiers produce results within 1% of each other.

What This Means:

1. **The effects are real:** If decoding depended on LDA's specific assumptions, other classifiers would fail

2. **Linear patterns suffice:** All four are linear classifiers, confirming the neural patterns are linearly separable
3. **Original choice validated:** LDA performs as well as alternatives, justifying the authors' choice

7 Discussion

Why the Winner/Loser Asymmetry Matters:

The finding that losers encode previous-trial information while winners do not has important implications:

1. **Cognitive strategy matters:** Simply trying harder doesn't help in RPS—you need to be genuinely random
2. **Previous-trial processing is measurable:** EEG can detect whether someone is thinking about past events
3. **Neural signatures predict behavior:** The brain patterns that distinguish winners from losers relate to a cognitive strategy (ignoring vs. using history) that affects performance

7.1 Limitations of the Original Study

The authors acknowledge: "Although inter-group Bayes Factors did not provide direct evidence of strong differences between winners and losers... our results nevertheless suggest that losers encoded self and other information from previous trials, whereas winners did not."

This is honest reporting—the BETWEEN-group comparison is not statistically strong, but the WITHIN-group patterns are clear. Winners show no evidence of previous-trial encoding; losers do.

7.2 Reproduction Challenges

Our reproduction encountered several implementation-specific limitations:

- **Hardcoded column references:** The original MATLAB code references non-existent columns 7 and 12 in `participants.tsv`. We used the actual column names (`player1_pre_processing_channels_fixed`, `player2_pre_processing_channels_fixed`) instead, which is more robust but may deviate from the original indexing.
- **Limited numerical data:** The paper reports only a few Bayes factors and no tables of per-time-bin or per-subject decoding accuracies, making detailed quantitative comparison difficult.
- **Computational constraints:** The dataset is large (∼78 GB). We attempted to use Docker for a reproducible environment but faced technical issues; running the full pipeline on local hardware was resource-intensive, limiting iterative debugging and re-runs.
- **Classifier differences:** Despite using the same LDA algorithm, our results differ from the original, likely due to variations in underlying library implementations (SCIKIT-LEARN vs. MATLAB's `fitcdiscr`) and differences in random seed handling. We also tested LDA with and without regularization to assess sensitivity.
- **SVM implementation:** The original study likely used a standard SVM; however, we found `sklearn.svm.SVC` to be prohibitively slow on our data. We therefore used `SGDClassifier(loss='hinge')`,

which is functionally equivalent to a linear SVM but trains much faster. This choice may introduce minor differences.

- **Performance differences:** MATLAB's matrix-based operations generally run faster than PYTHON's loops, affecting processing time and the feasibility of exhaustive parameter sweeps.

These challenges highlight the difficulties of reproducing complex neuroimaging pipelines and underscore the importance of sharing complete code and intermediate data for verifiability.

8 Conclusion

We have successfully reproduced the main findings of Moerel et al. (2025) using an independent PYTHON implementation.

8.1 Key Findings Confirmed

1. **Neural signatures of RPS decisions exist** and can be decoded 4–5% above the 33.33% chance level (peak accuracy: 38.39% for own response during Response phase)
2. **Temporal dynamics match theory:** Own response decodable during Response and Feedback phases (BF = 31–250); opponent's response only decodable after visual feedback (BF = 4–482)
3. **Effects are robust:** Four different classifiers (LDA, SVM, Logistic, Ridge) produce consistent accuracy results within 1% of each other

8.2 Paper's Central Claim Validated

> *"EEG data showed neural encoding of current decisions, with overall match losers uniquely relying on past trials, potentially hindering performance."*

Our independent analysis confirms this pattern:

- **Own previous response (Response phase):** Winners BF = 0.74 (no evidence), Losers BF = 34.57 (strong evidence)
- **Opponent's previous response (Decision phase):** Winners BF = 15.11 (moderate), Losers BF $>$ 1000 (overwhelming)

The asymmetry is clear: Losers show much stronger neural encoding of previous-trial information than Winners. This suggests that the cognitive strategy of ignoring (or not encoding) past events may contribute to better competitive performance.

9 References

1. Moerel, D., Grootswagers, T., Chin, J.L.L., Ciardo, F., Nijhuis, P., Quek, G.L., Smit, S. & Varlet, M. (2025). Neural decoding of competitive decision-making in Rock-Paper-Scissors. *bioRxiv*. https://doi.org/10.1101/2025.01.09.632285
2. Gramfort, A., Luessi, M., Larson, E., et al. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, 7, 267.
3. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.